

Lab 11.4 Part B – DevTools Debugging Report

Course: PRM392 - Mobile Application Development (Flutter)

Lab Module: Lab 11 — Testing & Debugging in Flutter (Taskly App)

Status: Completed

1. Potential Layout Issue: Text Overflow in ListTile Row

LAYOUT ISSUE

Description

In the TaskListScreen list layout, tasks are displayed inside a Row which acts as the main content structure for each task row. This Row contains a Checkbox, a Text widget for the task title, and an IconButton for deletions. When a user inputs an exceptionally long task title, the layout tries to expand horizontally without constraint boundaries. Because the screen has fixed width boundaries, this causes an **A-overflow (Right overflowed by X pixels)** rendering exception, displaying the striped hazard warning pattern on the viewport.

How Widget Inspector Detects It

- **Visual Boundary Highlight:** When inspecting with the Widget Inspector, a visual bounding box outlines the overflowing nodes, and the layout tree highlights nodes violating bounds in red.
- **Details Tree Constraints:** Selecting the Text node shows its width constraint is unbounded because it resides directly inside a row without width constraints.
- **Debug Paint Mode:** Enabling "Debug Paint" draws arrows and boundaries illustrating exactly where layout boxes exceed parent canvas limits.

Resolution:

Wrap the task Text widget inside an Expanded or Flexible widget. This forces the text layout width to align with the remaining horizontal space in the parent row, wrapping lines onto a new line or truncating with `TextOverflow.ellipsis`.

2. Potential Performance Issue: Unnecessary List View Rebuilds

PERFORMANCE ISSUE

Description

Whenever a task state is modified (like completion toggle or item addition/deletion), `setState()` is called in the main screen class. This rebuilds the entire `TaskListScreen` widget subtree, triggering a complete redraw of the `ListView.builder` container. In application scenarios with hundreds of tasks, redrawing every single `ListTile`, text node, and button during keystroke events or single checkbox updates degrades performance, causing frame drops (jank) and dropping rates below 60fps.

How Performance Timeline Detects It

- **Frame Rendering Graphs:** The Performance Timeline displays bar charts for each frame. If a rebuild operation takes longer than 16.6ms (60Hz) or 8.3ms (120Hz), the chart highlights it in red, indicating a frame drop.
- **Widget Rebuild Stats:** Using the rebuild tracker in DevTools shows abnormal build counts for list item widgets, indicating they are re-instantiated during unrelated state mutations.
- **CPU Profiler & GC Spikes:** The CPU flame chart outlines execution bottlenecks and garbage collection (GC) sweeps triggered by trash generation of redundant short-lived widgets.

Resolution:

- Extract child rows into standalone const widgets to prevent rebuilds if the inputs remain identical.
- Utilize localized build blocks like `ValueListenableBuilder`, `InheritedNotifier`, or state managers (Provider/Riverpod) to only update the specific item modified.

3. Conclusion: Benefits of Automated Testing & DevTools

Integrating tests with DevTools forms a robust quality framework:

- **Automated Testing:** Validates code logic and navigation flows programmatically, catching bugs before runtime deployment.
 - **DevTools:** Visualizes execution pipelines and widget structures, making it simple to track down constraints violations and frame rate drops.
-

PRM392 Showcase Application © 2026 - FPT University